

Bayesian Hidden Markov Models to Infer Latent Pitcher States

Garrett Lee, Trey Elder, Isaac Herman, Injun Lyo

1. Motivation

What Problem Does This Method Solve?

Think about the last time you tried to figure out if a friend was in a bad mood. You couldn't see inside their head, but you could observe their behavior - maybe they snapped at a question, skipped lunch, or sighed loudly several times. From those observations, you made an inference: they're probably in a bad mood. That intuition - inferring something hidden from things you *can* observe - is exactly what a Hidden Markov Model (HMM) does, just in a much more mathematical way.

To understand an HMM, it helps to first understand what a standard Markov model does. A Markov model is a probabilistic model describing a sequence of events with discrete states, where the probability of moving to the next state depends only on the current state - not on anything that came before it. As a simple example, suppose a friend named Bob does one of two activities each day: he either goes on a run or watches a movie, and tomorrow's activity depends only on today's activity. Each state has associated transition probabilities governing how likely he is to switch. If Bob ran today, there might be a 40% chance he runs tomorrow and a 60% chance he watches a movie. If he watched a movie today, there might be a 30% chance he watches again tomorrow and a 70% chance he runs.

		Bob's Transition Probabilities	
		Tomorrow's Activity	
		Run	Movie
Today's Activity	Movie	70%	30%
	Run	40%	60%

A Hidden Markov Model extends this by introducing an underlying state that drives the observations - but that state is never directly observed. In Bob's case, suppose his daily activity is actually determined by the weather, which can either be rainy or sunny. The weather evolves as a Markov process, but since Bob lives far away, we never observe the weather directly. We only see what activity he does. That unobserved weather state, which generates the observable activity outcomes, is the core of an HMM.

Every HMM has three components:

1. **Hidden states** - a discrete set of underlying conditions that evolve over time according to a Markov process. In the Bob example, these are the two weather states, rainy and sunny.
2. **Transition matrix** - the probabilities of moving from one hidden state to another. For instance, if it's raining today, there might be a 60% chance of rain tomorrow and a 40% chance of sun. These transition probabilities define how the hidden process evolves.
3. **Emission probabilities** - given the current hidden state, the probability of each observable outcome. In our example, if it's sunny there's a 60% chance Bob runs and a 40% chance he watches a movie; if it's raining, those probabilities reverse to 30% and 70%. The observations we see are *emitted* from the hidden state.

The key point is that we never observe the hidden states directly - we only ever see the emissions. The goal of the HMM is to work backwards: given the sequence of observations, what can we infer about the hidden states that generated them?

To make an HMM Bayesian, we place priors on the transition probabilities and emission probabilities, which are the parameters for the model. This allows us to incorporate domain knowledge or prior expectations into the model, and to quantify uncertainty. The Bayesian posterior we get also gives us full distributions over both model parameters and the hidden state sequences, rather than just point estimates.

Why Can't Standard Regression Handle It?

There are three fundamental reasons why standard regression is not suited for HMM problems. First, the cause is invisible. Regression requires you to explicitly specify the variables driving the variation in your outcome. In an HMM setting, that driver is the hidden state - which, by definition, we do not observe. There is no covariate to include in a regression model, because the thing generating the outcomes is unobserved.

Second, the independence assumption is violated. Linear regression assumes that observations are independent (given the covariates). However in sequential HMM data, we essentially assume the opposite to be true. Each observation is dependent on the hidden state, and each hidden state is dependent on the previous hidden state. This encodes severe autocorrelation into our data. HMMs are explicitly designed to handle this temporal dependence, but linear regression is not.

Third, the data-generating process itself changes. Regression produces a single, fixed relationship between predictors and outcomes. In an HMM setting, that relationship shifts depending on which hidden state we are in. This is modeled in an HMM through different emission probability distributions for each hidden state. However, in linear regression, we would be forced to model this using two multiple regressions, one for each hidden state. However, we cannot do this because we don't know which observations belong to which state. HMMs solve this by estimating the hidden states and the model parameters simultaneously.

Where Does It Show Up in Practice?

Hidden Markov Models have a long history of real-world application across many fields. The original and most widely-known use is speech recognition, where the hidden states correspond to phonemes - the fundamental units of sound in a word - and the observations are raw acoustic signals. Before the era of deep learning and transformer models, HMMs were the backbone of speech recognition systems for decades.

In genomics, HMMs have been used to infer whether sequences of DNA correspond to coding or non-coding regions, based on patterns in the nucleotide sequence. In finance, they are used to identify market regimes - bull markets, bear markets, high volatility periods - from observed price or return sequences. And in sports analytics, HMMs have been applied to model latent performance states, such as whether a basketball player is “hot” or “cold” during a game based on a sequence of their shot outcomes. This last application, sports analytics, is what we explore in the rest of this tutorial.

2. Model Specification

Likelihood

Independent Hidden Markov Sequences

We model pitch-level velocity data using an HMM, where each game is treated as an independent sequence. Let $y_{g,t}$ denote the observed four-seam fastball velocity for pitch t in game g , and let $z_{g,t} \in \{1, 2\}$ denote the corresponding latent state, representing *Baseline* and *Hot* performance regimes.

Conditional on the latent state, the observed velocity is modeled as:

$$y_{g,t} \mid z_{g,t} = k \sim \mathcal{N}(\mu_k, \sigma^2),$$

where μ_k is the state-specific mean velocity and σ^2 is a shared variance parameter, fixed using empirical variation from the 2024 season.

First-Order Markov State Evolution

The latent state evolves according to a first-order Markov process:

$$P(z_{g,t} = k \mid z_{g,t-1} = j) = \nu_{j,k},$$

where ν is a 2×2 transition matrix. This implies that the current state depends only on the immediately preceding state, not on the full history of past states. This assumption induces temporal persistence in the latent performance state.

Initial State and Modeling Choices

At the beginning of each game, the latent state is drawn from an initial distribution:

$$z_{g,1} \sim \text{Categorical}(\pi),$$

where π is a vector of initial state probabilities. Treating games as independent sequences reflects the assumption that latent performance states do not carry over across games, which is reasonable given rest periods between starts.

We restrict attention to four-seam fastballs and use velocity as the sole observable. This choice ensures interpretability and prevents the latent states from capturing pitch type rather than underlying performance condition.

Priors

We place independent priors on the emission means μ , the transition matrix ν , and the initial state distribution π .

Emission Means

The state-specific means are assigned Normal priors:

$$\mu_k \sim \mathcal{N}(m_k, s^2),$$

where m_k is derived from historical pre-2024 velocity data and $s = 2$ mph provides a diffuse prior. Specifically, historical velocities are split at the 75th percentile to define approximate baseline and hot regimes, yielding prior centers near 96 mph and 99 mph.

Importantly, this does not imply that 75% of pitches are assumed to belong to the baseline state. Rather, this split serves only as a heuristic for centering the priors:

$$\text{Before observing 2024 data, we believe } \mu_1 \approx 96 \text{ and } \mu_2 \approx 99,$$

with substantial uncertainty allowed by the prior variance.

Transition Probabilities

Each row of the transition matrix is assigned a Dirichlet prior:

$$\nu_j \sim \text{Dirichlet}(\alpha_{j1}, \alpha_{j2}),$$

with concentration parameters chosen to favor self-transitions (here we set $\alpha_{jj} = 8$, $\alpha_{jk} = 1$ for $j \neq k$). This reflects the belief that pitcher condition is persistent rather than rapidly switching between states.

Initial State Distribution

The initial state distribution is also assigned a Dirichlet prior:

$$\pi \sim \text{Dirichlet}(\gamma_1, \gamma_2),$$

with greater weight placed on the baseline state, reflecting the expectation that pitchers are more likely to begin a game in a typical performance condition.

Role of the Dirichlet Prior

The Dirichlet distribution is a natural prior for probability vectors that must be non-negative and sum to one. It can be viewed as a multivariate generalization of the Beta distribution. Its conjugacy with multinomial-type counts is particularly useful in this setting: once latent states are sampled, posterior updates reduce to adding observed transition counts to prior parameters, enabling efficient Gibbs sampling.

Hierarchical Structure

The model has a hierarchical structure defined by its generative process. At the highest level are global parameters shared across all games: the emission means μ , transition probabilities ν , and initial distribution π . At the intermediate level are the latent state sequences $\{z_{g,t}\}$, which evolve within each game according to the Markov process. At the lowest level are the observed velocities $y_{g,t}$, generated conditionally on the latent states.

This defines the generative hierarchy:

$$(\mu, \nu, \pi) \rightarrow z_{g,t} \rightarrow y_{g,t}.$$

It is important to emphasize that this is a **latent variable hierarchy**, not a multilevel random effects model. The model does not include group-specific parameters drawn from higher-level distributions and therefore does not perform partial pooling across games or players.

Key Modeling Assumptions

Several assumptions underlie the model specification.

First, the model assumes exactly two latent performance states. This imposes a binary structure on pitcher condition. If the true process exhibits more nuanced or continuous variation, the model will compress this variation into two regimes. Increasing the number of states could improve flexibility but may reduce interpretability and increase the risk of overfitting.

Second, the model assumes time-homogeneous transitions:

$$P(z_{g,t} \mid z_{g,t-1}) \text{ is constant over } t.$$

In practice, transition dynamics may depend on pitch count, fatigue, or inning, which are not explicitly modeled here.

Third, the emission distribution is assumed to be Normal:

$$y_{g,t} \mid z_{g,t} = k \sim \mathcal{N}(\mu_k, \sigma^2),$$

implying symmetric, unimodal variation within each state. Real velocity data may exhibit skewness, heavy tails, or within-game trends, which this specification cannot capture.

Finally, the model fixes the variance σ^2 across states and uses a single observable (velocity). This simplifies estimation but restricts the model to capturing only one dimension of performance variation.

Because HMM states are label-invariant under the likelihood, the model imposes an ordering constraint:

$$\mu_1 < \mu_2,$$

so that state 1 corresponds to *Baseline* and state 2 corresponds to *Hot*. This ensures interpretability and consistency across posterior samples.

3. Applied Example

Data Description and Context

We apply a Bayesian Hidden Markov Model to pitch-level velocity data for **Gerrit Cole**, a right-handed starting pitcher for the New York Yankees. Cole was selected for his high four-seam fastball usage rate and consistent full seasons of starts (30+ per year), giving us roughly 1,000–2,000 four-seam fastballs per season to work with.

Emission metric: We model a single observable: `release_speed` (pitch velocity in mph) for four-seam fastballs only. Restricting to one pitch type is essential, since mixing pitch types would cause the HMM to discover pitch type as the hidden state rather than performance condition. Velocity is the most straightforward and interpretable signal of a pitcher’s condition. Elevated velocity indicates peak effort, while declining velocity often signals fatigue.

Hidden states: The model posits two latent states, *Baseline* and *Hot*, representing the pitcher’s true performance mode at each pitch. The state is unobserved; we only see the noisy velocity measurements that each state generates.

Game structure: Pitches are grouped by game start, and each game is treated as an **independent sequence**. The hidden state resets at the beginning of each game. This reflects the reality that games are separated by 4–5 days of rest, so carry-over fatigue between games is not modeled.

Data source: Pitch-level data come from MLB Statcast via the `baseballr` package, covering the 2019–2024 seasons. The raw data are fetched week-by-week and cached locally to avoid repeated API calls.

Load Data

We pull pitch-level Statcast data for Gerrit Cole across the 2019–2024 MLB seasons. Data are retrieved in weekly chunks from Baseball Savant and saved locally so subsequent runs read from the cache rather than re-querying the API. All pitch types are retained at this stage; filtering to four-seam fastballs happens in the cleaning step.

Below is a sample of the raw data that was collected:

Pitcher	Date	Season	Pitch Type	Velocity (mph)	Spin Rate (rpm)
Cole, Gerrit	2019-04-03	2019	KC	83.5	3181
Cole, Gerrit	2019-04-03	2019	CH	87.1	1667
Cole, Gerrit	2019-04-03	2019	FF	97.9	2605
Cole, Gerrit	2019-04-03	2019	KC	83.8	2959
Cole, Gerrit	2019-04-03	2019	SL	89.4	2777
Cole, Gerrit	2019-04-03	2019	FF	98.8	2634

Clean Data

We filter the raw data to four-seam fastballs for the 2024 season and restructure it into the format expected by the HMM. Within each season, games are numbered sequentially by date. Within each game, pitches are sorted by inning, then at-bat, then position within the at-bat to recover true chronological order, and renumbered 1, 2, 3, ... as a single game-level sequence.

Season	Pitcher	Game	Pitch	Velocity (mph)
2024	Cole, Gerrit	1	1	94.2
2024	Cole, Gerrit	1	2	94.1
2024	Cole, Gerrit	1	3	97.0
2024	Cole, Gerrit	1	4	96.6
2024	Cole, Gerrit	1	5	95.7
2024	Cole, Gerrit	1	6	95.2

Fit Model

Prior Selection

We need to center the Normal prior on each emission mean somewhere reasonable. As a starting point, we use Cole’s historical velocity data from all seasons prior to 2024, split at the 75th percentile: the lower 75% of pitches approximate the Baseline state and the upper 25% approximate the Hot state. The mean of each group becomes the center of the corresponding prior. This is a rough heuristic. A 75/25 split doesn’t correspond to any structural assumption about how often Cole is in each state. The intent is simply to give the sampler a sensible starting region rather than an uninformative flat prior. With thousands of 2024 pitches available, the likelihood will dominate and the prior centers matter little in practice.

State	Mean (mph)	SD (mph)	N (pitches)
Baseline	96.71	1.03	5388
Hot	98.85	0.56	1756

The three sets of model parameters each receive their own prior.

For the **emission means** (μ_k), we use Normal priors centered on the empirical group means above, with a standard deviation of 2 mph. This is wide enough that the posterior is driven by the 2024 data, but anchored close enough to realistic velocity values that the sampler does not waste time exploring implausible regions.

State	Prior mean (mph)	Prior SD (mph)
Baseline	96	2
Hot	99	2

For the **transition matrix rows** (ν_k), we use Dirichlet priors that place more weight on self-transitions than on switching. A concentration parameter of 8 on the diagonal versus 1 on the off-diagonal reflects the prior belief that a pitcher’s state tends to persist from one pitch to the next — states are sticky, not erratic.

Current state	Stay (Dirichlet param)	Switch (Dirichlet param)	Prior P(stay)
Baseline	8	1	~89%
Hot	8	1	~89%

For the **initial state distribution** (π), we use a Dirichlet prior that puts most of its weight on starting in the Baseline state at the beginning of each game. The asymmetry reflects the reasonable assumption that a pitcher is more likely to begin a start in a typical rather than elevated performance mode.

State	Dirichlet param	Prior P(start here)
Baseline	6	~75%
Hot	2	~25%

Gibbs Sampler

The HMM has three sets of unknown parameters: the emission means μ , the transition matrix ν , and the initial state distribution π , plus the hidden state sequence itself. Because the full joint posterior over all of these is intractable, we use a **Gibbs sampler**: a Markov chain Monte Carlo algorithm that cycles through each block of unknowns in turn, drawing from its conditional distribution given the current values of everything else. After a burn-in period, the chain’s samples approximate the joint posterior.

The hidden state sequences are a fourth unknown, but they are not a parameter in the usual sense. They are latent variables that must be integrated over. Because conditioning on the state sequences makes the parameter updates simple conjugate draws, we include them in the sampler and alternate between updating states and updating parameters.

Each iteration of the sampler performs four steps:

Step 1: Sample hidden states (FFBS). Given the current parameter values, we sample a complete hidden state sequence for each game independently using the **Forward-Filter Backward-Sample** algorithm. In the forward pass, we compute $\alpha_{t,k}$, the joint probability of the observed velocities up through pitch t and being in state k at pitch t . This recursion sweeps forward through the game, accumulating evidence. In the backward pass, we draw states from right to left: the last pitch’s state is sampled from the normalized final forward probabilities, and each earlier state is sampled conditional on the state drawn immediately after it. This produces an exact draw from $p(\mathbf{z} \mid \mathbf{y}, \mu, \nu, \pi)$.

Because games are independent sequences, hidden states reset at the start of each game, with no carry-over between starts. FFBS is run separately for each of the G games per iteration.

Step 2: Sample transition probabilities (ν). Given the sampled state sequences, we count how many times each state-to-state transition occurs across all games. These counts, added to the Dirichlet prior parameters, form the posterior hyperparameters for each row of ν . Each row is then drawn independently from its Dirichlet full conditional.

Step 3: Sample initial state distribution (π). We count how many games begin in each state (i.e., look at the first sampled state of each game). Adding these counts to the Dirichlet prior on π gives the posterior hyperparameters, from which we draw a new π .

Step 4: Sample emission means (μ_k). Given the state assignments for every pitch, we pool all pitches assigned to state k across all games and compute their sample mean and count. Combined with the Normal prior, this gives a Normal full conditional for μ_k , from which we draw directly. This step is repeated for each state.

We also apply a **label-switching correction** after each iteration. Because the states $\{1, 2\}$ are exchangeable under the likelihood, the sampler can freely permute their labels between iterations. Without correction, samples from different chains would not be comparable. We fix this by enforcing a canonical ordering: after each draw, we sort the states by their emission means ($\mu_1 < \mu_2$), relabeling all state assignments, both rows and columns of ν , and π accordingly.

To assess convergence and reduce dependence on starting values, we run **four independent chains** from different initializations and discard the first half of each chain as burn-in. We reshape the pitch-level data into Y , a list of length G where Y_g is the ordered vector of four-seam fastball velocities for game g . We also fix σ to the empirical standard deviation of all 2024 velocities, a shared, known variance assumed by the model.


```

# Reshape cleaned data into list-of-games format expected by the HMM
Y <- split(filtered_gerrit_cole$release_speed, filtered_gerrit_cole$game_number)
Y <- lapply(Y, unname)
sigma <- sd(unlist(Y))

run_gibbs_chain <- function(
  Y, sigma, prior_nu, prior_pi, prior_mu_mean,
  prior_mu_var, init_nu, init_pi, init_mu, numsamp
) {
  K <- length(init_mu)
  G <- length(Y)
  Tg <- sapply(Y, length)
  total_pitches <- sum(Tg)

  # Storage matrices
  mu.samp <- matrix(NA, nrow = numsamp, ncol = K)
  nu.samp <- matrix(NA, nrow = numsamp, ncol = K * K)
  pi.samp <- matrix(NA, nrow = numsamp, ncol = K)
  X.samp <- matrix(NA, nrow = numsamp, ncol = total_pitches)

  # Current parameter values
  curnu <- init_nu
  curpi <- init_pi
  curmu <- init_mu
  X <- lapply(Tg, function(t) rep(NA, t))

  for (iter in 1:numsamp) {
    #### Block 1: FFBS - sample hidden states per game ####
    for (g in 1:G) {
      alpha <- matrix(NA, nrow = Tg[g], ncol = K)

      # Forward pass
      for (i in 1:K) {
        alpha[1, i] <- curpi[i] * dnorm(
          Y[[g]][1],
          curmu[i], sigma
        )
      }
      if (Tg[g] > 1) {
        for (t in 2:Tg[g]) {
          for (i in 1:K) {
            alpha[t, i] <- 0
            for (j in 1:K) {
              alpha[t, i] <- alpha[t, i] +
                alpha[t - 1, j] * curnu[j, i] * dnorm(
                  Y[[g]][t],
                  curmu[i], sigma
                )
            }
          }
        }
      }
    }
  }
}

```

```

# Backward sampling
prob_vec <- alpha[Tg[g], ] / sum(alpha[Tg[g], ])
X[[g]][Tg[g]] <- sample(1:K, size = 1, prob = prob_vec)
if (Tg[g] > 1) {
  for (t in (Tg[g] - 1):1) {
    for (i in 1:K) {
      prob_vec[i] <- alpha[t, i] * curnu[i, X[[g]][t + 1]]
    }
    prob_vec <- prob_vec / sum(prob_vec)
    X[[g]][t] <- sample(1:K, size = 1, prob = prob_vec)
  }
}

#### Block 2: Sample transition matrix ####
Nmat <- matrix(0, nrow = K, ncol = K)
for (g in 1:G) {
  for (t in 2:Tg[g]) {
    j <- X[[g]][t - 1]
    k <- X[[g]][t]
    Nmat[j, k] <- Nmat[j, k] + 1
  }
}
for (j in 1:K) {
  for (k in 1:K) {
    curnu[j, k] <- rgamma(1, shape = Nmat[j, k] + prior_nu[j, k], rate = 1)
  }
  curnu[j, ] <- curnu[j, ] / sum(curnu[j, ])
}

#### Block 3: Sample initial distribution ####
init_count <- rep(0, K)
for (g in 1:G) {
  init_count[X[[g]][1]] <- init_count[X[[g]][1]] + 1
}
for (j in 1:K) {
  curpi[j] <- rgamma(1, shape = init_count[j] + prior_pi[j], rate = 1)
}
curpi <- curpi / sum(curpi)

#### Block 4: Sample emission means ####
for (j in 1:K) {
  obs_j <- unlist(lapply(1:G, function(g) Y[[g]][X[[g]] == j]))
  n_j <- length(obs_j)
  if (n_j > 0) {
    Y_bar_j <- mean(obs_j)
    post_prec <- 1 / prior_mu_var + n_j / sigma^2
    post_mean <- (prior_mu_mean[j] / prior_mu_var
      + n_j * Y_bar_j / sigma^2) / post_prec
    curmu[j] <- rnorm(1, mean = post_mean, sd = sqrt(1 / post_prec))
  } else {
    curmu[j] <- rnorm(1, mean = prior_mu_mean[j], sd = sqrt(prior_mu_var))
  }
}

```

```

    }

    ##### Store samples #####
    mu.samp[iter, ] <- curmu
    nu.samp[iter, ] <- as.vector(curnu)
    pi.samp[iter, ] <- curpi
    X.samp[iter, ] <- unlist(X)
  }

  list(mu = mu.samp, nu = nu.samp, pi = pi.samp, X = X.samp, game_lengths = Tg)
}

fix_label_switching <- function(mu_mat, nu_mat, pi_mat, X_mat) {
  n_iter <- nrow(mu_mat)
  for (i in 1:n_iter) {
    if (mu_mat[i, 1] > mu_mat[i, 2]) {
      # Swap mu
      temp_mu <- mu_mat[i, 1]
      mu_mat[i, 1] <- mu_mat[i, 2]
      mu_mat[i, 2] <- temp_mu

      # Swap pi
      temp_pi <- pi_mat[i, 1]
      pi_mat[i, 1] <- pi_mat[i, 2]
      pi_mat[i, 2] <- temp_pi

      # Swap nu (column-major vectorization: nu11, nu21, nu12, nu22)
      temp_nu11 <- nu_mat[i, 1]
      temp_nu21 <- nu_mat[i, 2]
      temp_nu12 <- nu_mat[i, 3]
      temp_nu22 <- nu_mat[i, 4]
      nu_mat[i, 1] <- temp_nu22
      nu_mat[i, 4] <- temp_nu11
      nu_mat[i, 2] <- temp_nu12
      nu_mat[i, 3] <- temp_nu21

      # Relabel X
      current_X <- X_mat[i, ]
      X_mat[i, current_X == 1] <- 2
      X_mat[i, current_X == 2] <- 1
    }
  }
  list(mu = mu_mat, nu = nu_mat, pi = pi_mat, X = X_mat)
}

```

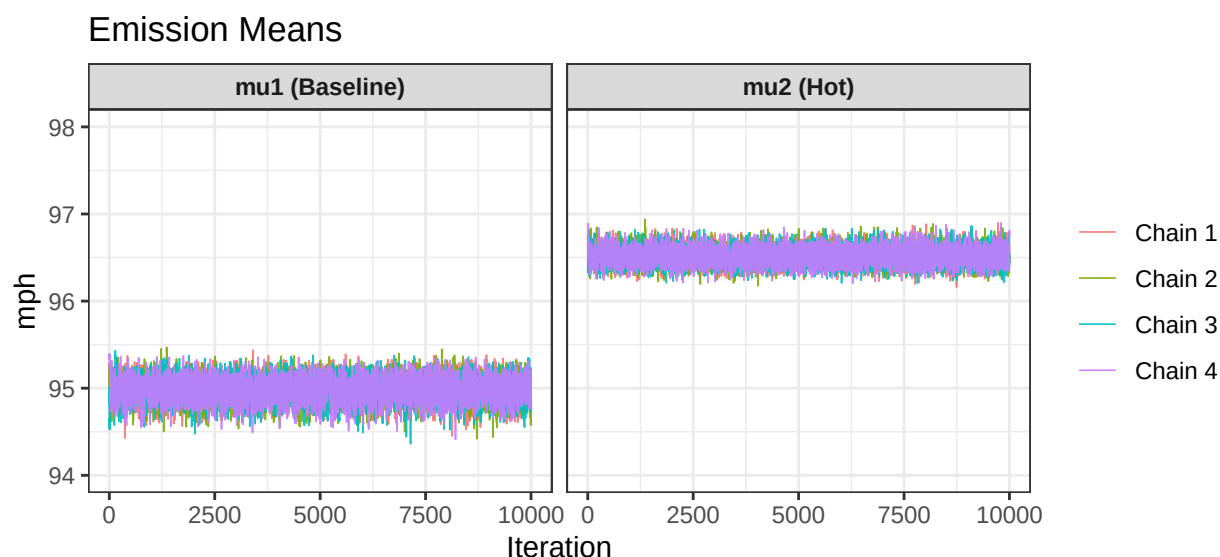
MCMC Diagnostics

We ran four independent Gibbs chains from different starting values to check convergence. Trace plots show the sampled value of each parameter at every iteration. Four well-mixed chains that overlap and wander around the same region, rather than drifting apart or getting stuck, are the primary visual signal that the sampler has converged.

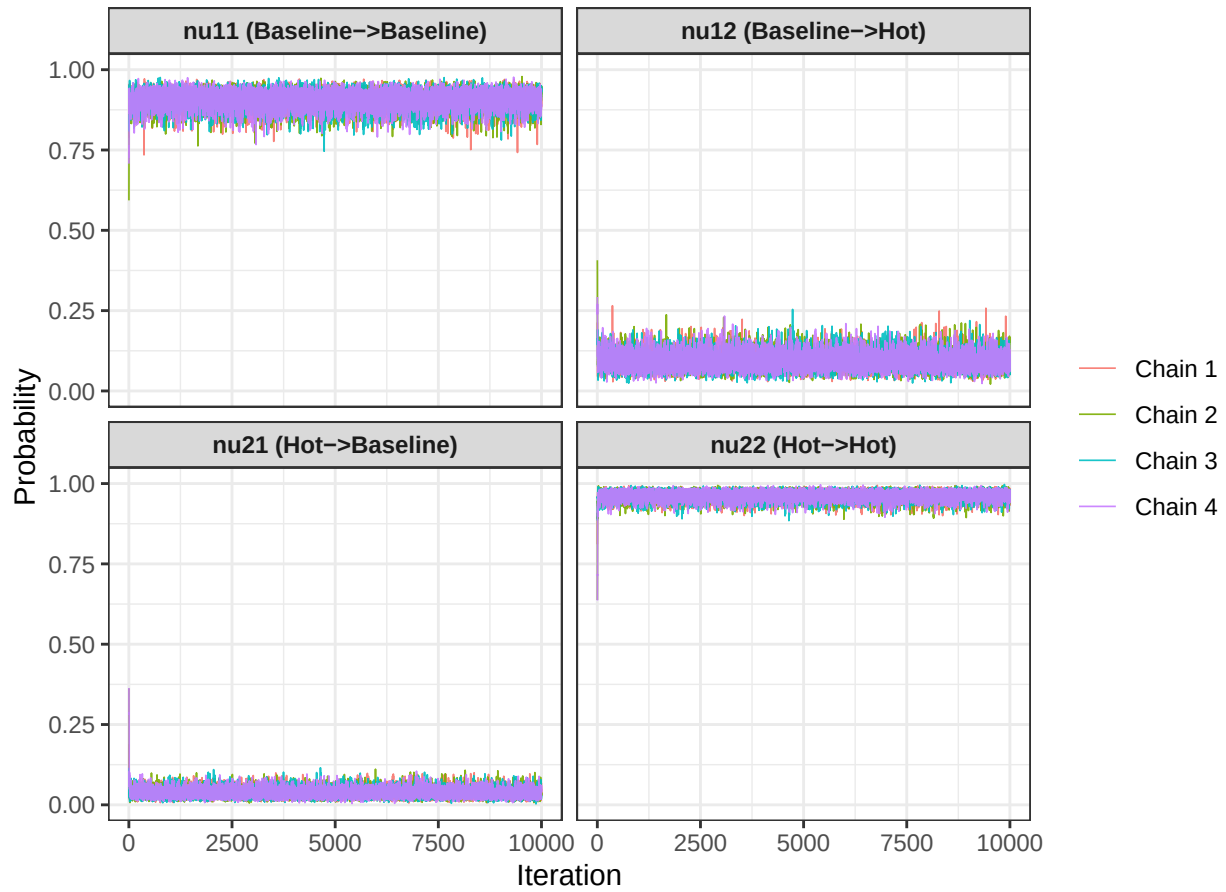
We monitor all sampled scalar parameters across three figures: emission means (μ_1, μ_2), all four transition probabilities ($\nu_{11}, \nu_{12}, \nu_{21}, \nu_{22}$), and the initial state distribution (π_1, π_2).

Initial Samples

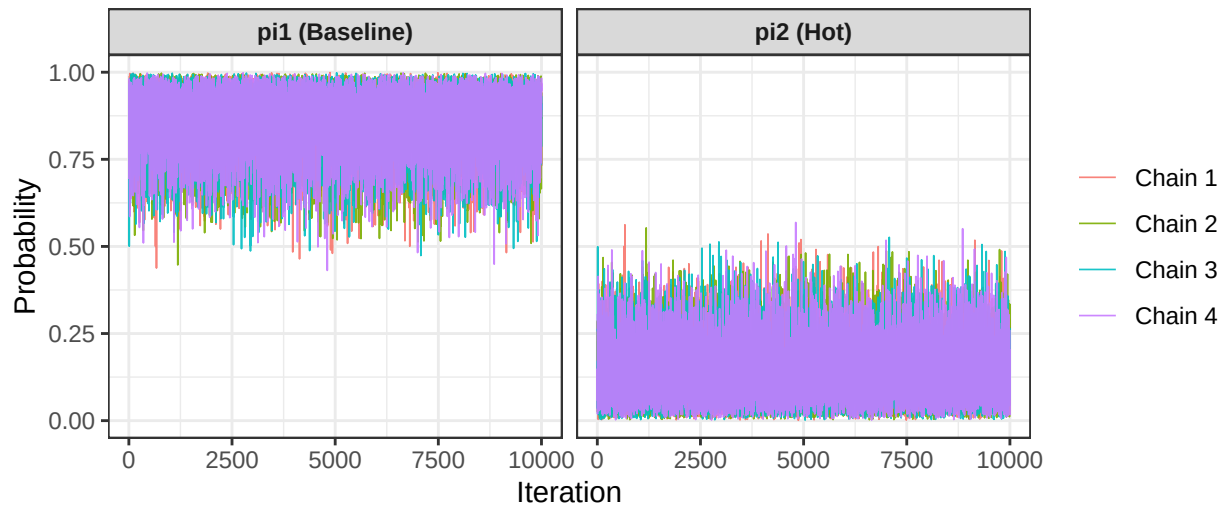
These are the raw trace plots from the Gibbs sampler: every draw from all 10,000 iterations, with no post-processing applied. Consecutive samples from a Gibbs chain are not independent: each draw depends on the one before it, so the chain can exhibit autocorrelation, where successive values move slowly and the effective number of independent samples is much smaller than the raw count.



Transition Probabilities

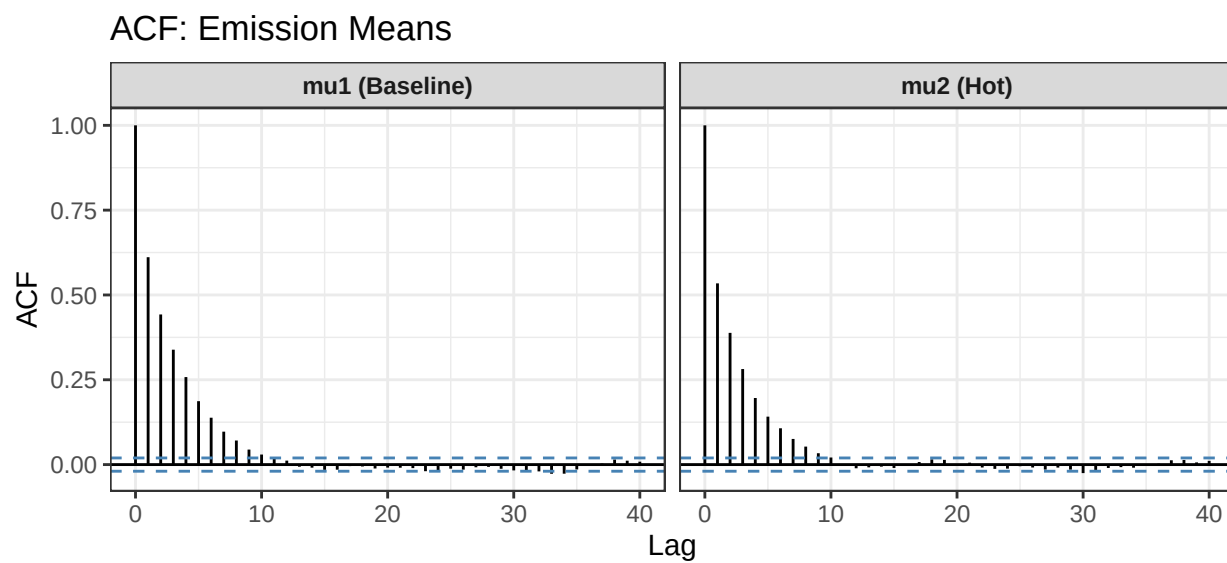


Initial State Distribution

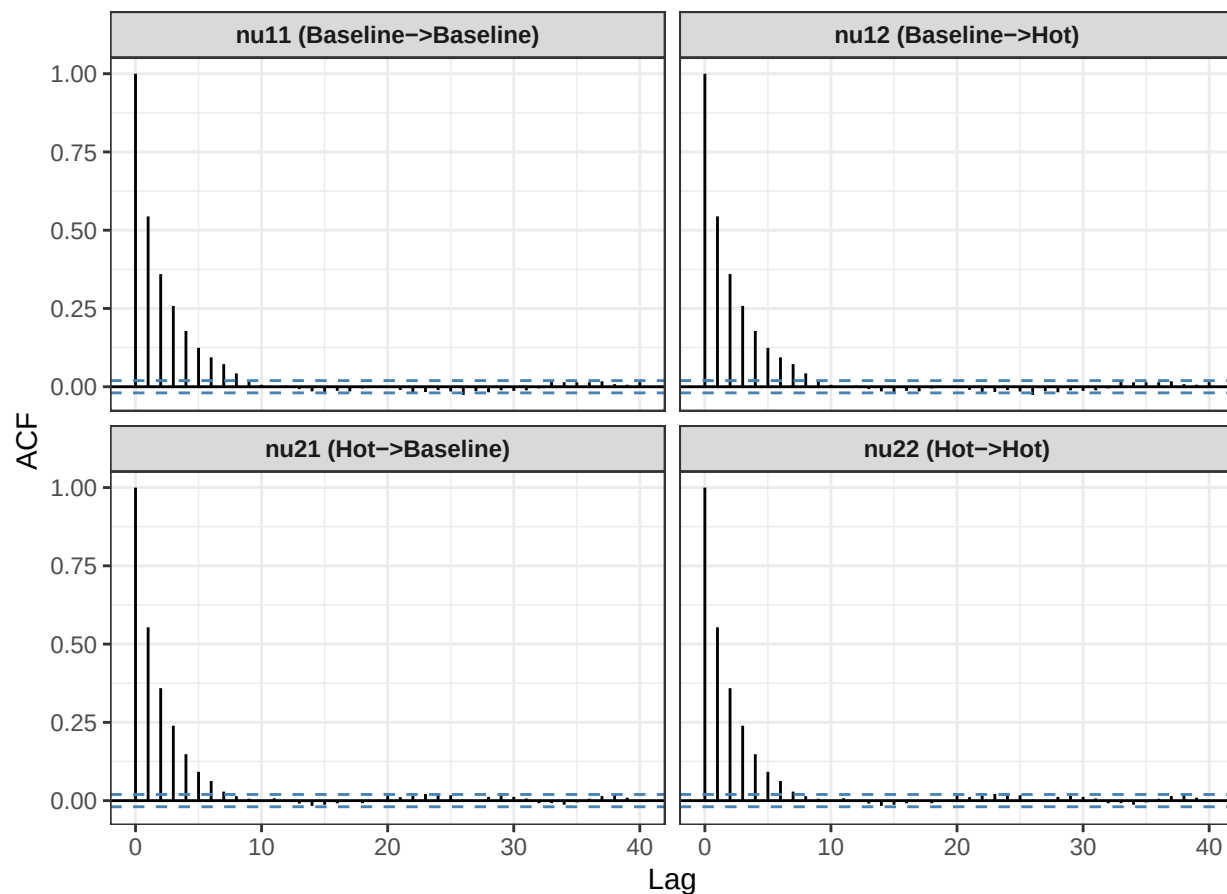


Thinned Samples

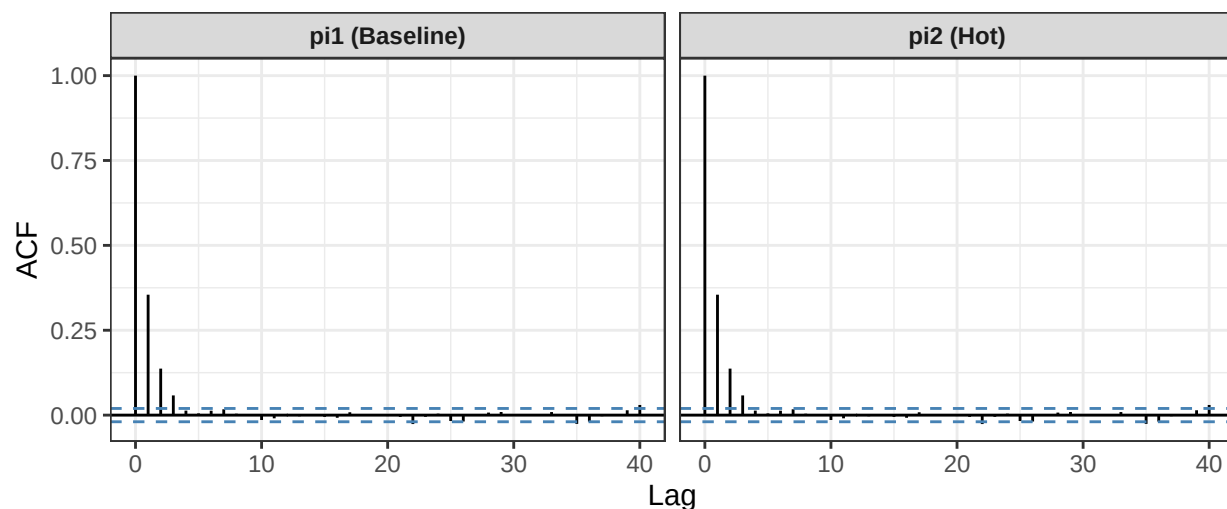
To diagnose autocorrelation, we compute the **autocorrelation function (ACF)** for each parameter. The ACF measures the correlation between a chain's samples at lag k , that is, how similar a draw is to the draw k steps before it. High autocorrelation at moderate lags (say, lags 1–10) indicates that the chain is moving slowly through the posterior and that many consecutive samples carry nearly the same information. The lag at which the ACF first drops close to zero tells us the minimum spacing needed for approximately independent draws.



ACF: Transition Probabilities



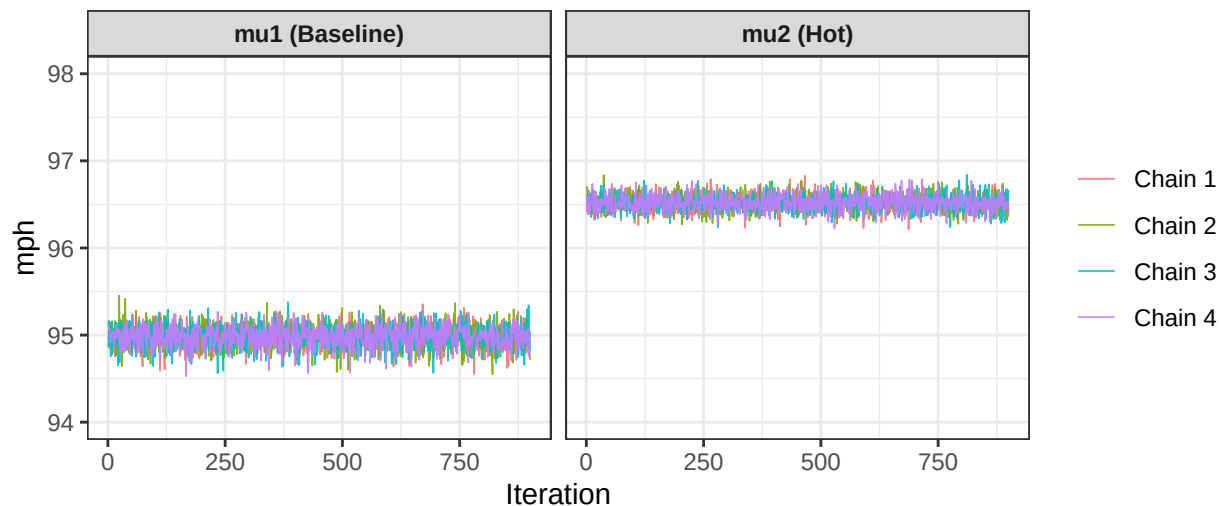
ACF: Initial State Distribution



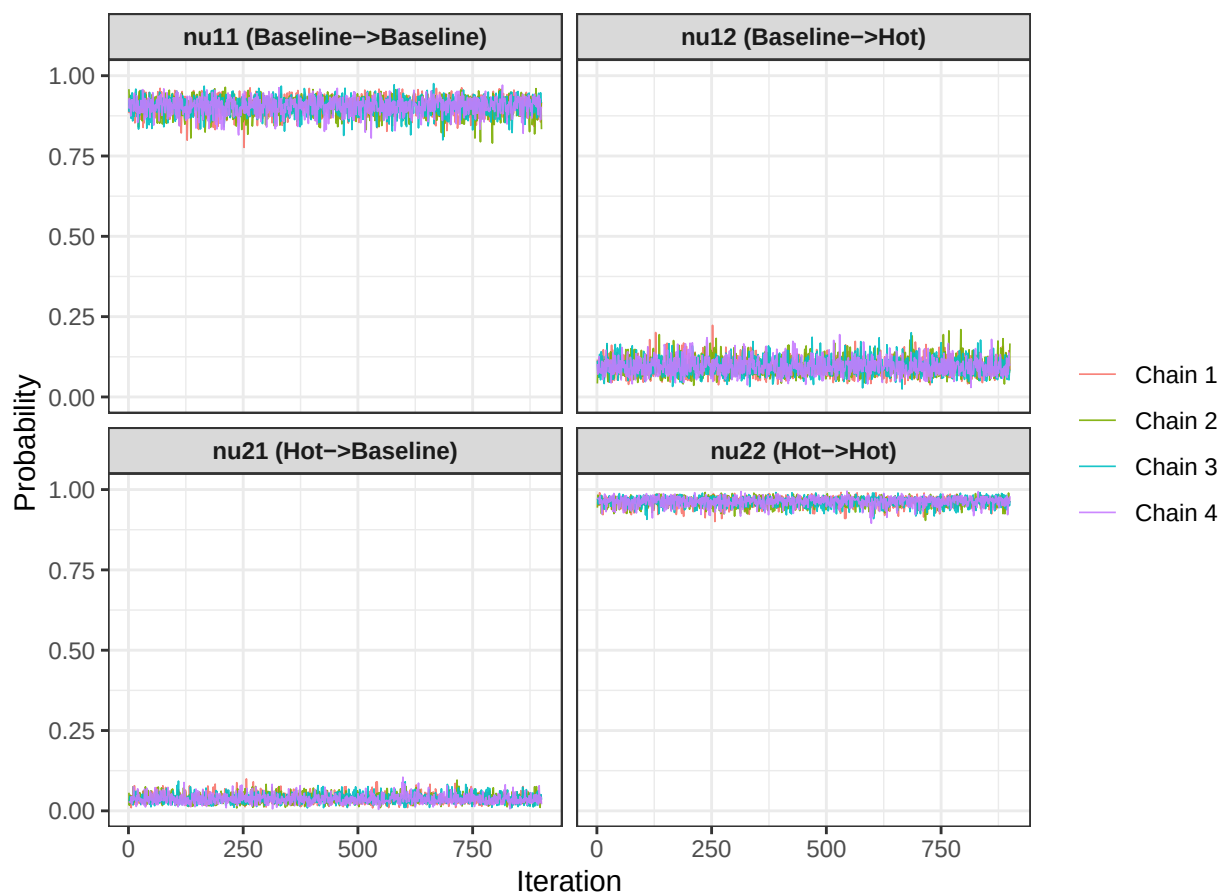
Based on the ACF, we thin the chain by keeping only every k -th sample, where k is chosen to be at or beyond the lag where autocorrelation becomes negligible. Thinning reduces the effective sample size but produces a set of draws that are much closer to independent, which makes posterior summaries and credible intervals more reliable. The trace plots below show the sampled values after thinning.

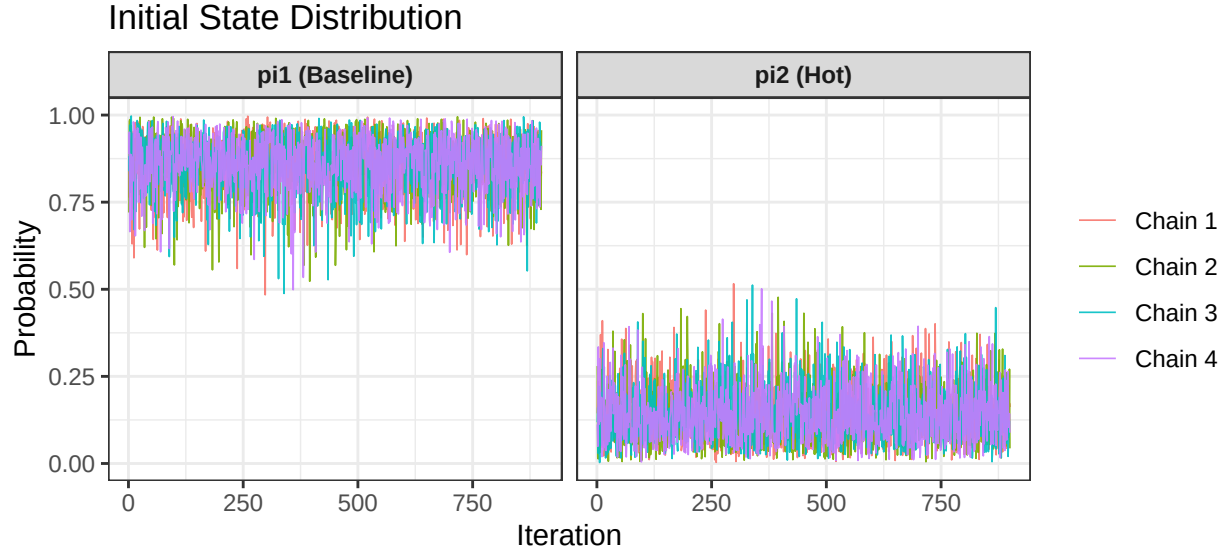
The ACF plots above show that autocorrelation across all parameters decays fully by roughly lag 8–10. We discard the first 1,000 iterations as burn-in to ensure the starting values have no influence on inference, then thin by keeping every 10th sample. This leaves 900 draws per chain (3,600 combined), which is enough for stable posterior summaries.

Emission Means



Transition Probabilities





Posterior Summaries

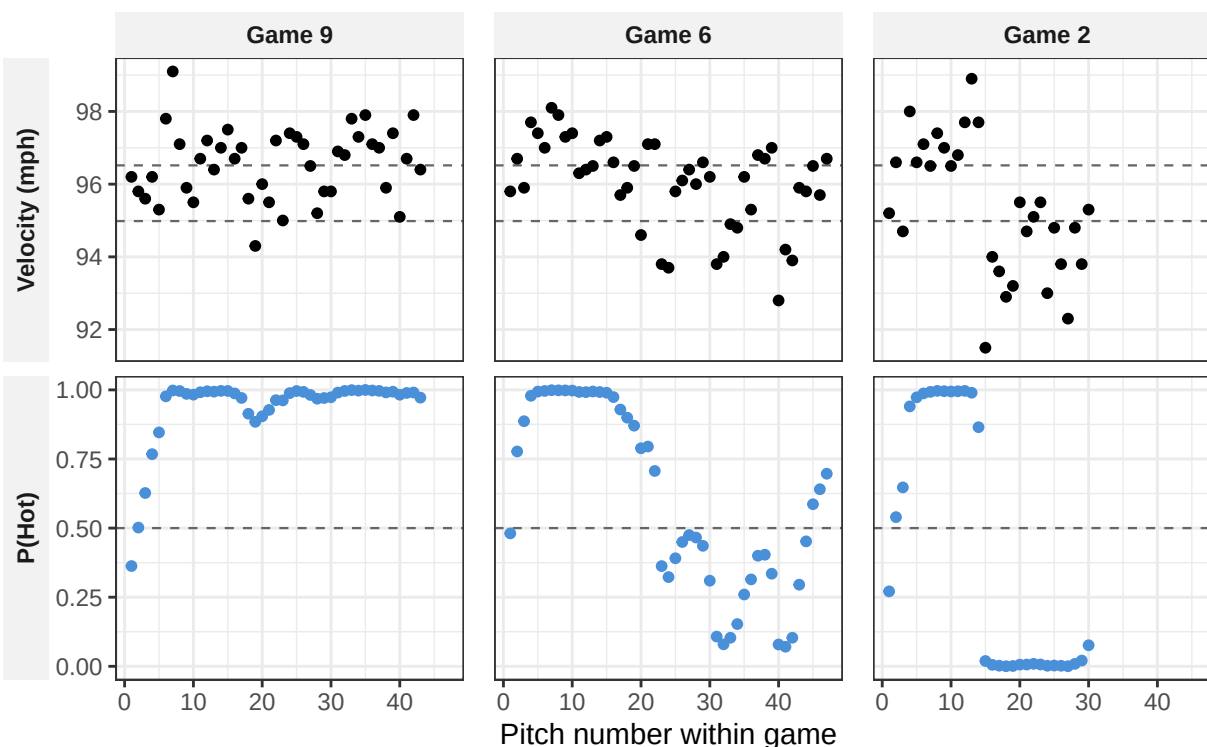
Table 1: Posterior summaries from 3,600 thinned draws (4 chains $\times$ 900 samples). CI bounds are the 10th and 90th percentiles.

Parameter	Mean	SD	10%	90%
mu1 (Baseline mean, mph)	94.984	0.126	94.821	95.139
mu2 (Hot mean, mph)	96.517	0.092	96.401	96.638
nu11 (Baseline $\rightarrow$ Baseline)	0.905	0.027	0.870	0.937
nu12 (Baseline $\rightarrow$ Hot)	0.095	0.027	0.063	0.130
nu21 (Hot $\rightarrow$ Baseline)	0.038	0.014	0.022	0.056
nu22 (Hot $\rightarrow$ Hot)	0.962	0.014	0.944	0.978
pi1 (Pr. start Baseline)	0.860	0.082	0.744	0.956
pi2 (Pr. start Hot)	0.140	0.082	0.044	0.256

Visualizations

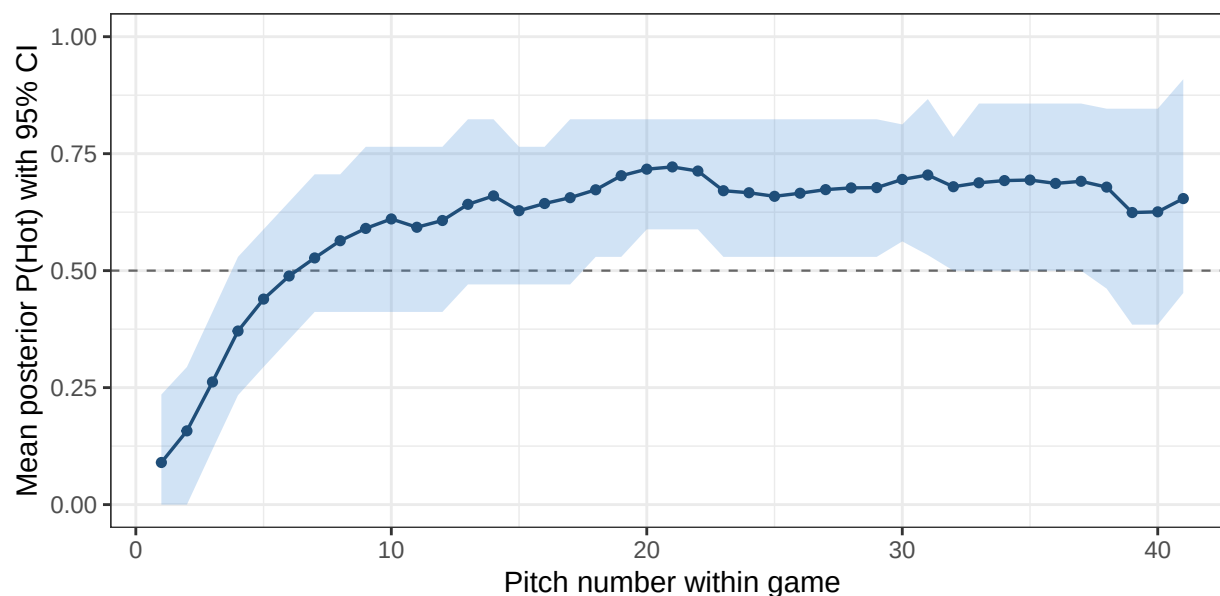
The plot below pairs observed fastball velocity (top row) with the posterior $P(\text{Hot})$ for each pitch (bottom row), one column per game. The three games are chosen to span the within-game $\text{sd}(P(\text{Hot}))$ distribution (minimum, median, and maximum) so reading left-to-right tours the full range of HMM behavior, from starts where the model is entirely confident to starts where it is actively switching states. Reading each column *vertically* is the central diagnostic: a pitch near $\hat{\mu}_{\text{Hot}}$ should line up with a point near 1 directly below it, and a pitch near $\hat{\mu}_{\text{Baseline}}$ with a point near 0.

How this lines up with reality: Cole’s 2024 was cut in half by an elbow injury that sidelined him through mid-June, which is why “game 2” in this plot is already June 25 and “game 9” lands on August 10. The two starts bracket his comeback arc. Game 2 was his second outing back, a Subway Series loss in which he struggled and next-day coverage ran with headlines like *“Gerrit Cole’s Subway Series clobbering comes with stark velocity concerns”* (Joyce, *NY Post*, 2024-06-25). Game 9 was the opposite: six scoreless innings and a season-high ten strikeouts against Texas, widely reported as a return to ace form. The HMM never saw a box score or a headline, it only saw release speeds, but the pitch-level posterior on these two starts already encodes what the writers were saying at the time, which is a useful sanity check that the latent “Hot” state is picking up something real.



The figure below asks a simpler question: *on average, how hot is Cole as a function of pitch index within a start?* For each pitch position t , and separately in every Gibbs draw, we compute the fraction of games (among those that reached pitch t) that were in the Hot state at that pitch. The posterior mean of that quantity gives the central line; the 2.5/97.5 quantiles across draws give a 95% credible interval on the across-game mean.

The curve tells a clean warm-up story. The first few pitches sit deep in the Baseline state, the estimate climbs monotonically and crosses 0.5 early in the outing, and from roughly the middle of a start onward it plateaus above 0.5 for the rest of the game. This is the familiar pattern of a pitcher getting loose in the first inning, now showing up in the pitch-level posterior. The credible interval is tightest through the middle of a start, where dozens of games contribute, and widens at both ends where fewer games support the estimate.



Posterior Predictive Checks and Model Improvement

Here, we run a posterior predictive check on the observed velocity data (not the hidden states). This code takes one posterior draw from the parameters, simulates a fake season from the HMM using those parameters, computes summary statistics on that fake season, and repeats many times.

Here, we actually assess model fit using a posterior predictive check by simulating replicated velocity sequences from the fitted HMM. For each posterior draw of the parameters μ , ν , and π , we generate a full “season” of pitch velocities that preserves the observed game lengths. This produces a distribution of synthetic data representing what the model believes the data should look like.

For both the observed data and each replicated dataset, we compute a set of summary statistics: overall mean velocity, overall standard deviation, mean within-game range, and mean lag-1 autocorrelation. The inclusion of lag-1 autocorrelation is important because it captures pitch-to-pitch persistence within games, which is a defining feature of the HMM’s Markov structure.

Model fit is evaluated by comparing the observed statistics to the posterior predictive distributions. Specifically, we examine whether the observed values fall near the center of the replicated distributions and within their 95% intervals.

Table 2: Posterior predictive check based on replicated velocity sequences (Original Model).

Statistic	Observed	Replicated_Mean	Replicated_2.5	Replicated_97.5
Overall mean velocity	95.918	95.904	95.641	96.168
Overall velocity SD	1.284	1.482	1.386	1.579
Mean within-game range	4.982	6.184	5.742	6.694
Mean within-game lag-1 autocorrelation	0.431	0.129	0.040	0.221

Posterior Predictive Check (Visual Diagnostics)

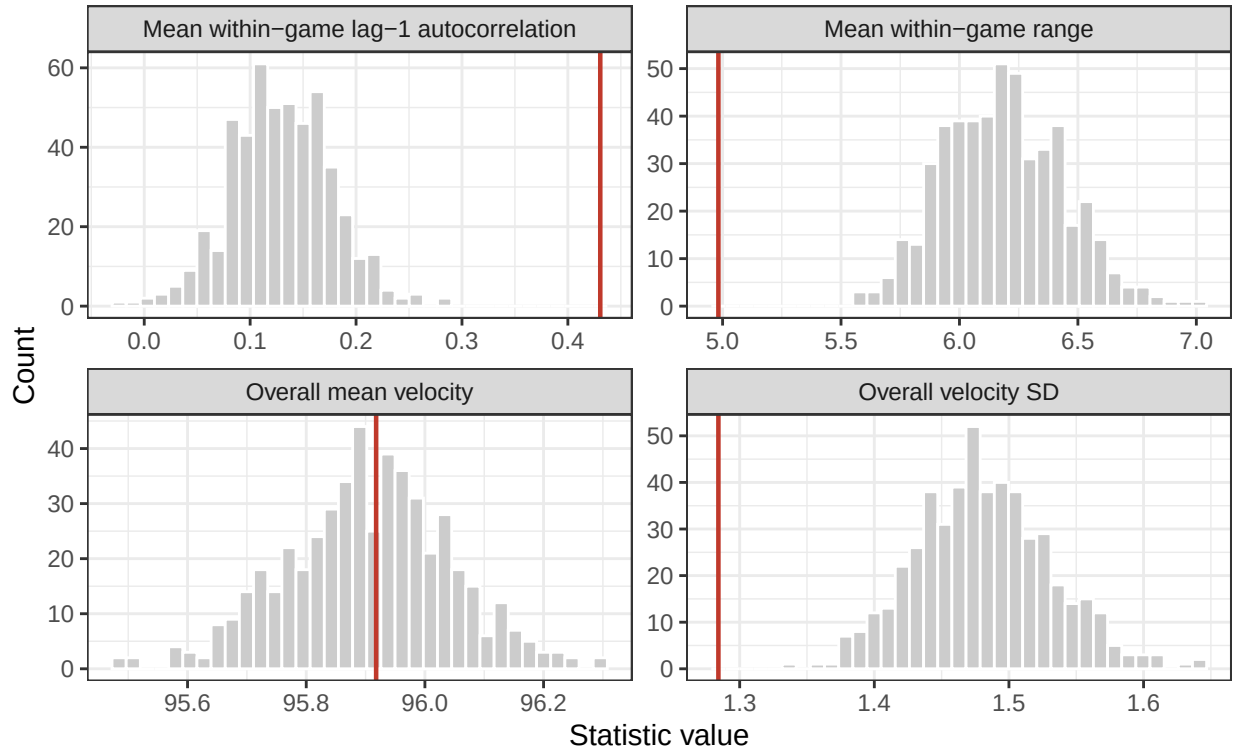
Next, we visualize the posterior predictive distributions of each statistic using histograms. Each panel shows the distribution of a statistic computed across replicated datasets, with the observed value overlaid as a vertical reference line.

The results indicate inadequate model performance. The **overall mean velocity** is well captured, as the observed value lies near the center of the replicated distribution. However, the model performs poorly on higher-order structure. The observed **lag-1 autocorrelation** lies far outside the replicated range, indicating that the model substantially underestimates temporal persistence. Similarly, the observed **within-game range** and **overall standard deviation** fall in the tails of their respective distributions, showing that the model misrepresents the magnitude and structure of variability.

Overall, while the model captures the central tendency of the data, it fails to reproduce key features of variability and temporal dependence, indicating a misspecification in the underlying data-generating assumptions.

Posterior Predictive Checks

Histogram = replicated seasons from posterior draws; red line = observed statistic



Posterior Predictive Check (Density Overlay)

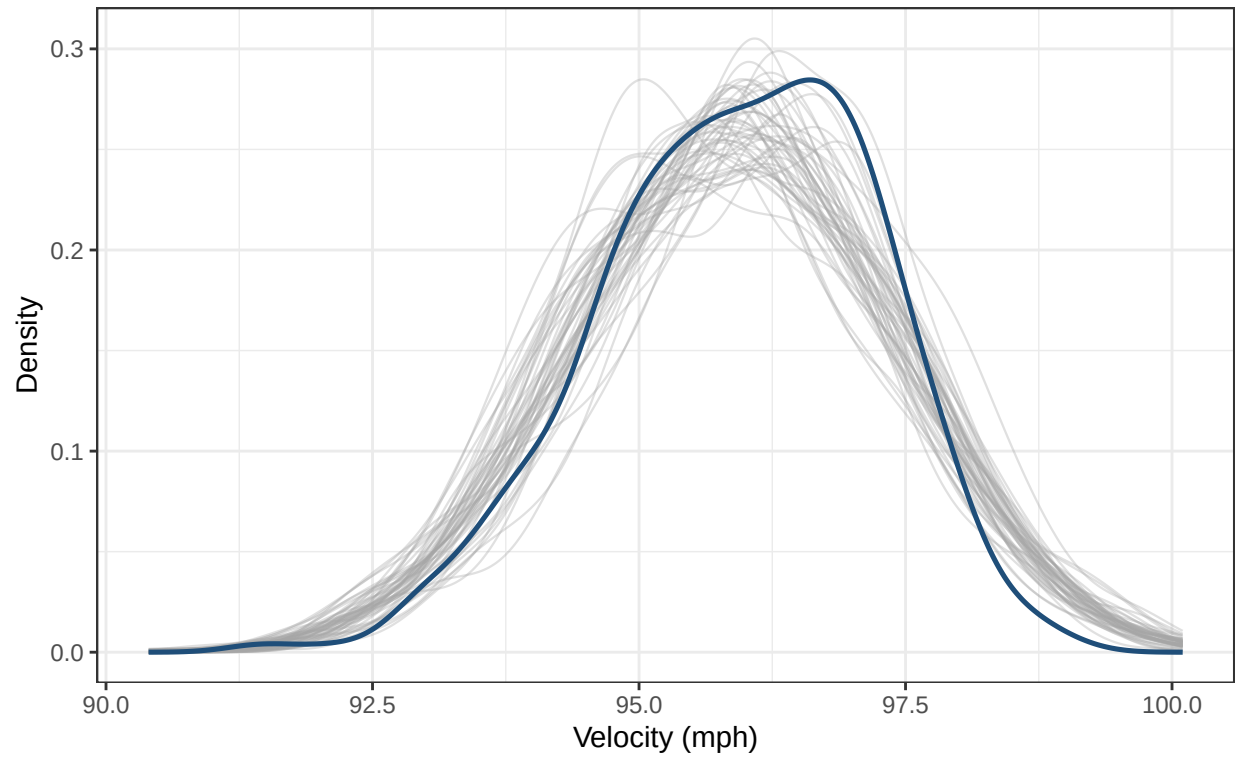
We further assess model fit by comparing the **full distribution of observed velocities** to replicated datasets drawn from the posterior predictive distribution. For a subset of posterior draws, we simulate entire seasons of pitch velocities and compute kernel density estimates for each replicated dataset. These are overlaid against the observed density.

The model captures the **overall location and general shape** of the velocity distribution reasonably well, as the observed curve lies within the envelope of replicated densities. However, systematic discrepancies remain. The observed distribution appears **slightly sharper and more asymmetric**, while the replicated densities are smoother and more diffuse. This suggests the model struggles to reproduce finer features of the distribution, such as tail behavior and subtle skewness.

Overall, the model adequately matches the central tendency, but it fails to fully capture the **distributional structure of velocities**.

Observed vs Posterior Predictive Velocity Density

Blue = observed season; gray = replicated seasons



Revised Model Specification

To address the lack of fit in the original model, we extend the emission distribution in two ways.

First, we allow the latent states to have **state-specific variances**, so that variability may differ between the Baseline and Hot regimes. Second, we explore adding a **linear within-game pitch-number trend** by constructing a centered covariate $x_{g,t} = t - \bar{t}_g$, where t indexes pitch number within game g . Centering the pitch index improves interpretability and reduces confounding between the intercepts and the trend term.

Under the revised specification, the observation model becomes

$$y_{g,t} \mid z_{g,t} = k \sim \mathcal{N}(\mu_k + \beta x_{g,t}, \sigma_k^2),$$

where μ_k is the state-specific mean velocity, β is the linear pitch-number effect shared across states, and σ_k^2 is the variance for state k . The hidden state process remains a first-order Markov chain with transition matrix ν , and the initial state of each game is still drawn from π . Thus, the revision changes the emission model but preserves the latent Markov structure.

The Gibbs sampler is expanded accordingly. Hidden states are still sampled game-by-game using FFBS, but the forward probabilities now depend on the trend-adjusted state means and state-specific standard deviations. Conditional on the sampled states, the sampler then updates: (i) the transition matrix ν , (ii) the initial distribution π , (iii) the state means μ_1, μ_2 , (iv) the linear trend coefficient β , and (v) the state-specific variances σ_1^2, σ_2^2 . The variance parameters are updated from inverse-gamma full conditionals, while β is updated from a Normal full conditional.

This revised model is designed to separate three distinct sources of structure in the data: **state differences in mean velocity**, **within-state variability**, and **systematic within-game change**. Relative to the original model, it reduces the need to explain gradual warm-up or fatigue patterns entirely through abrupt state switching, while also allowing the two latent regimes to differ in dispersion as well as location.

```
run_gibbs_chain_trend_var <- function(  
  Y, Xcov,  
  prior_nu, prior_pi,  
  prior_mu_mean, prior_mu_var,  
  prior_beta_mean, prior_beta_var,  
  prior_sig2_a, prior_sig2_b,  
  init_nu, init_pi, init_mu, init_beta, init_sig2,  
  numsamp  
) {  
  K <- length(init_mu)  
  G <- length(Y)  
  Tg <- sapply(Y, length)  
  total_pitches <- sum(Tg)  
  
  # storage  
  mu.samp <- matrix(NA, nrow = numsamp, ncol = K)  
  beta.samp <- numeric(numsamp)  
  sig2.samp <- matrix(NA, nrow = numsamp, ncol = K)  
  nu.samp <- matrix(NA, nrow = numsamp, ncol = K * K)  
  pi.samp <- matrix(NA, nrow = numsamp, ncol = K)  
  X.samp <- matrix(NA, nrow = numsamp, ncol = total_pitches)  
  
  # Current values  
  curnu <- init_nu  
  curpi <- init_pi  
  curmu <- init_mu
```

```

curbeta <- init_beta
cursig2 <- init_sig2
Xstate <- lapply(Tg, function(t) rep(NA_integer_, t))

for (iter in 1:numsamp) {
  # FFBS for hidden states
  for (g in 1:G) {
    alpha <- matrix(NA_real_, nrow = Tg[g], ncol = K)

    # t = 1
    for (k in 1:K) {
      alpha[1, k] <- curpi[k] * dnorm(
        Y[[g]][1],
        mean = curmu[k] + curbeta * Xcov[[g]][1],
        sd = sqrt(cursig2[k])
      )
    }
    alpha[1, ] <- alpha[1, ] / sum(alpha[1, ])

    # t >= 2
    if (Tg[g] > 1) {
      for (t in 2:Tg[g]) {
        for (k in 1:K) {
          alpha[t, k] <- dnorm(
            Y[[g]][t],
            mean = curmu[k] + curbeta * Xcov[[g]][t],
            sd = sqrt(cursig2[k])
          ) * sum(alpha[t - 1, ] * curnu[, k])
        }
        alpha[t, ] <- alpha[t, ] / sum(alpha[t, ])
      }
    }

    # backward sample
    Xstate[[g]][Tg[g]] <- sample(1:K, size = 1, prob = alpha[Tg[g], ])

    if (Tg[g] > 1) {
      for (t in (Tg[g] - 1):1) {
        prob_vec <- alpha[t, ] * curnu[, Xstate[[g]][t + 1]]
        prob_vec <- prob_vec / sum(prob_vec)
        Xstate[[g]][t] <- sample(1:K, size = 1, prob = prob_vec)
      }
    }
  }
}

# sample transition matrix
Nmat <- matrix(0, nrow = K, ncol = K)
for (g in 1:G) {
  if (Tg[g] > 1) {
    for (t in 2:Tg[g]) {
      j <- Xstate[[g]][t - 1]
      k <- Xstate[[g]][t]
      Nmat[j, k] <- Nmat[j, k] + 1
    }
  }
}

```



```

    }
  }
}

for (j in 1:K) {
  row_draw <- rgamma(K, shape = Nmat[j, ] + prior_nu[j, ], rate = 1)
  curnu[j, ] <- row_draw / sum(row_draw)
}

# sample initial distribution
init_count <- rep(0, K)
for (g in 1:G) {
  init_count[Xstate[[g]][1]] <- init_count[Xstate[[g]][1]] + 1
}
pi_draw <- rgamma(K, shape = init_count + prior_pi, rate = 1)
curpi <- pi_draw / sum(pi_draw)

# Flatten data for parameter updates
y_all <- unlist(Y)
x_all <- unlist(Xcov)
z_all <- unlist(Xstate)

# sample state means mu_k
for (k in 1:K) {
  idx <- which(z_all == k)
  n_k <- length(idx)

  if (n_k > 0) {
    y_star <- y_all[idx] - curbeta * x_all[idx]
    post_prec <- 1 / prior_mu_var + n_k / cursig2[k]
    post_mean <- (prior_mu_mean[k] / prior_mu_var +
      sum(y_star) / cursig2[k]) / post_prec
    curmu[k] <- rnorm(1, mean = post_mean, sd = sqrt(1 / post_prec))
  } else {
    curmu[k] <- rnorm(1, mean = prior_mu_mean[k], sd = sqrt(prior_mu_var))
  }
}

# Sample linear trend beta
mu_by_obs <- curmu[z_all]
sig2_by_obs <- cursig2[z_all]
y_tilde <- y_all - mu_by_obs

post_prec_beta <- 1 / prior_beta_var + sum((x_all^2) / sig2_by_obs)
post_mean_beta <- (prior_beta_mean / prior_beta_var
  + sum(x_all * y_tilde / sig2_by_obs)) / post_prec_beta

curbeta <- rnorm(1, mean = post_mean_beta, sd = sqrt(1 / post_prec_beta))

# sample state-specific variances sigma_k^2
for (k in 1:K) {
  idx <- which(z_all == k)
  n_k <- length(idx)

```

```

    if (n_k > 0) {
      resid_k <- y_all[idx] - curmu[k] - curbeta * x_all[idx]
      post_a <- prior_sig2_a + n_k / 2
      post_b <- prior_sig2_b + sum(resid_k^2) / 2
    } else {
      post_a <- prior_sig2_a
      post_b <- prior_sig2_b
    }

    cursig2[k] <- 1 / rgamma(1, shape = post_a, rate = post_b)
  }

  # store draws
  mu.samp[iter, ] <- curmu
  beta.samp[iter] <- curbeta
  sig2.samp[iter, ] <- cursig2
  nu.samp[iter, ] <- as.vector(curnu)
  pi.samp[iter, ] <- curpi
  X.samp[iter, ] <- z_all
}

list(
  mu = mu.samp,
  beta = beta.samp,
  sig2 = sig2.samp,
  nu = nu.samp,
  pi = pi.samp,
  X = X.samp,
  game_lengths = Tg
)
}

```

Label-Switching Correction

As in the original HMM, the model is invariant to relabeling the latent states, so the sampler may arbitrarily swap state labels across iterations. For interpretability, we enforce a consistent ordering by requiring the lower-mean state to be labeled as state 1 and the higher-mean state as state 2.

At each iteration, if $\mu_1 > \mu_2$, the function swaps all state-dependent quantities: the means μ , variances σ^2 , initial probabilities π , transition matrix ν , and the latent state assignments.

This does not affect the model fit but ensures posterior summaries are comparable across iterations/chains.

Prior Specification

We retain the same Dirichlet priors for the transition matrix ν and initial state distribution π , with parameters chosen to favor **state persistence** (e.g., (8, 1)) and a higher probability of starting in the Baseline state (e.g., (6, 2)). These reflect the belief that pitcher condition evolves gradually rather than switching rapidly.

For the emission means μ_k , we keep Normal priors with variance 4 (i.e., SD = 2 mph), which are **weakly informative**—wide enough to let the data dominate, but centered in realistic velocity ranges to stabilize estimation.

We introduce a new parameter β for the linear trend, with prior $\beta \sim \mathcal{N}(0, 1)$. This is centered at zero to reflect no assumed trend, while allowing sufficient flexibility for moderate within-game changes.

For the state-specific variances σ_k^2 , we use inverse-gamma priors $\sigma_k^2 \sim \text{IG}(2, 1)$. These are weakly informative, placing most mass on realistic variance values while allowing each state to differ in dispersion.

Finally, we run four chains with **dispersed initial values** for μ , ν , π , and σ^2 , and initialize $\beta = 0$. This ensures robust convergence by exploring different regions of the parameter space before settling into the posterior.

Posterior Summaries

As in the original model, we discard the first 1,000 iterations as burn-in and thin the remaining draws by keeping every 10th sample. This reduces autocorrelation and yields a set of posterior draws that are more independent. The thinned draws are then pooled across all four chains for inference.

Posterior summaries are reported for the revised parameter set: the state means μ_1, μ_2 , the linear trend coefficient β , the state-specific variances σ_1^2, σ_2^2 , the transition probabilities ν , and the initial state distribution π . For each parameter, we report the posterior mean, posterior standard deviation, and the 10th and 90th percentiles to summarize location, uncertainty, and posterior spread.

- **State means** (μ_1, μ_2): The Baseline and Hot states remain clearly separated (approx. 95.0 vs. 96.8 mph), confirming the model still captures meaningful differences in performance levels.
- **Trend** (β): The posterior mean is small and close to zero, with uncertainty spanning zero. This suggests only weak evidence of a systematic within-game velocity trend after accounting for state switching.
- **State-specific variances** (σ_1^2, σ_2^2): The Baseline state has higher variability than the Hot state. This indicates that elevated performance is more consistent, while baseline pitching is noisier.
- **Transition probabilities** (ν): Both states remain highly persistent (approx. 88–91% chance of staying), reinforcing the interpretation of sustained performance regimes rather than rapid switching.
- **Initial state** (π): The model strongly favors starting in the Baseline state, consistent with prior expectations and earlier results.

Overall, the new parameters refine interpretation: the model now separates **level (means)**, **variability (state-specific variance)**, and **gradual changes (trend)**, rather than forcing all structure into state switching.

Table 3: Posterior summaries from the HMM with state-specific variances and a linear pitch-number trend.

Parameter	Mean	SD	10%	90%
mu1 (Baseline mean, mph)	95.014	0.117	94.859	95.161
mu2 (Hot mean, mph)	96.759	0.091	96.641	96.873
beta (pitch-number trend)	0.005	0.006	-0.004	0.013
sigma1^2 (Baseline variance)	1.077	0.102	0.947	1.207
sigma2^2 (Hot variance)	0.710	0.078	0.610	0.813
nu11 (Baseline -> Baseline)	0.877	0.029	0.839	0.912
nu12 (Baseline -> Hot)	0.123	0.029	0.088	0.161
nu21 (Hot -> Baseline)	0.091	0.022	0.063	0.121
nu22 (Hot -> Hot)	0.909	0.022	0.879	0.937
pi1 (Pr. start Baseline)	0.884	0.072	0.786	0.966

Parameter	Mean	SD	10%	90%
pi2 (Pr. start Hot)	0.116	0.072	0.034	0.214

Revised HMM Convergence Diagnostics

The revised model shows strong convergence across all parameters. All $\hat{R}$ values are essentially 1.00–1.001, indicating that the four chains have mixed well and are sampling from the same posterior distribution.

ESS is large for all parameters, generally in the thousands, suggesting low autocorrelation and reliable posterior estimates. In particular, the variance and initial state parameters exhibit very high ESS, indicating especially stable inference.

Overall, these diagnostics provide strong evidence that the Gibbs sampler has converged and that the posterior summaries can be trusted.

Table 4: Convergence diagnostics for the revised HMM.

variable	mean	sd	rhat	ess_bulk	ess_tail
mu1	95.014	0.117	1.001	2426.987	5514.249
mu2	96.758	0.089	1.001	2647.290	6439.506
beta	0.004	0.006	1.001	3452.083	9992.030
sig2_1	1.078	0.101	1.000	9975.598	17065.651
sig2_2	0.711	0.079	1.001	4399.006	11565.679
nu11	0.877	0.029	1.001	4340.253	12849.499
nu21	0.091	0.023	1.000	6074.745	14809.789
nu12	0.123	0.029	1.001	4340.253	12849.499
nu22	0.909	0.023	1.000	6074.745	14809.789
pi1	0.886	0.071	1.000	21001.752	32450.910
pi2	0.114	0.071	1.000	21001.752	32450.910

Posterior Predictive Simulation (Revised Model)

For the revised model, posterior predictive datasets are generated in the same way as before, but using the expanded emission structure. Each replicated game is simulated from the fitted HMM with state-specific means, a linear pitch-number trend, and state-specific variances:

$$y_{g,t} \mid z_{g,t} = k \sim \mathcal{N}(\mu_k + \beta x_{g,t}, \sigma_k^2).$$

Replicated seasons are then formed by simulating each game at its observed length and concatenating resulting velocity sequences.

Similar to the original PPC, model fit is evaluated using overall mean velocity, overall standard deviation, mean within-game range, and mean lag-1 autocorrelation. This assesses whether the revised model can reproduce not only the central tendency of the data, but also its variability and pitch-to-pitch persistence.

Interpretation of Posterior Predictive Check (Revised Model)

The revised model shows substantial improvement in overall fit. Here, observed mean and standard deviation fall well within the posterior predictive intervals, indicating the model accurately captures the central tendency and overall variability of velocities.

The within-game range is also reasonably well matched, suggesting improved modeling of dispersion within games.

However, the model still underestimates temporal dependence. The observed lag-1 autocorrelation lies above the replicated interval, indicating that real pitch velocities are more persistent than the model can reproduce. While the added trend and state-specific variances improve fit, some temporal structure remains unaccounted for.

It is important to note that under the revised model, latent states are no longer interpreted as raw **Baseline** and **Hot** regimes alone. Instead, they represent performance regimes *after accounting for* a linear within-game trend and state-specific variability. This improves fit, but it also makes interpretation less direct, because smooth changes over a game and differences in variance can now explain part of the same structure that was previously attributed entirely to state switching.

Table 5: Posterior predictive check for the HMM with state-specific variances and a linear pitch-number trend.

Statistic	Observed	Replicated_Mean	Replicated_2.5	Replicated_97.5
Overall mean velocity	95.918	95.920	95.629	96.176
Overall velocity SD	1.284	1.284	1.185	1.372
Mean within-game range	4.982	5.175	4.769	5.628
Mean within-game lag-1 autocorrelation	0.431	0.279	0.187	0.374

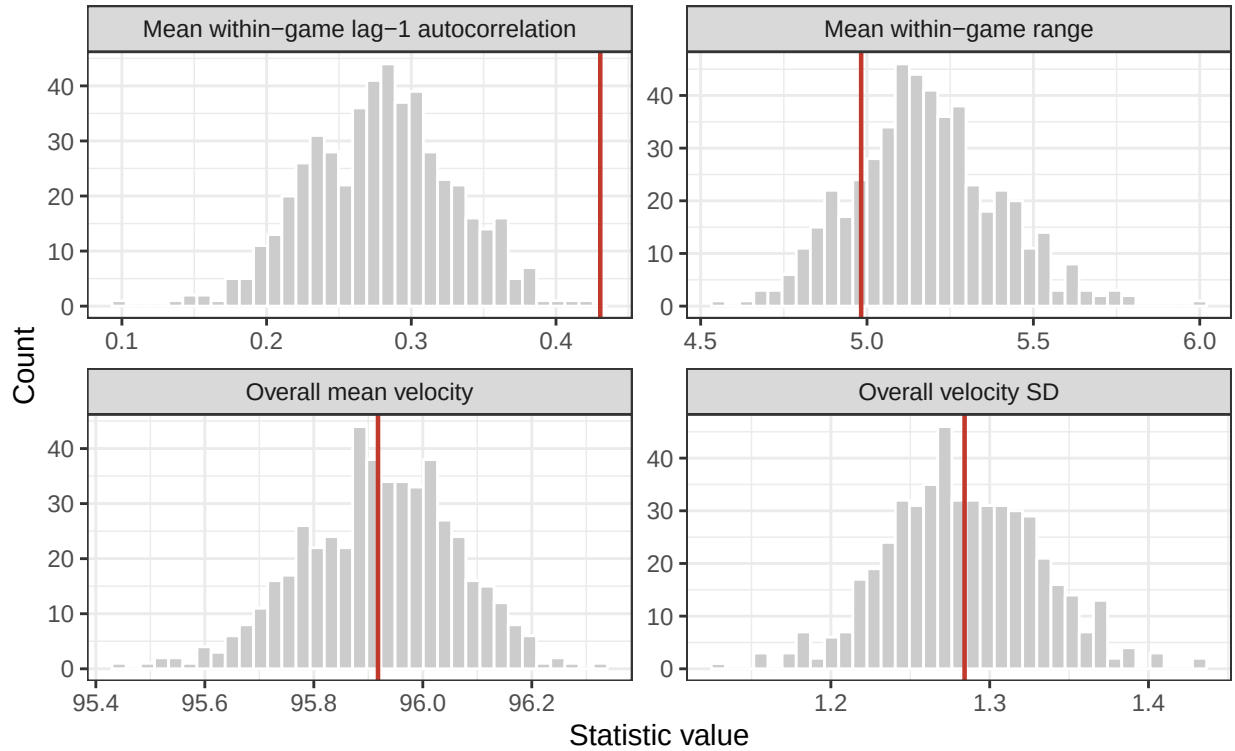
Posterior Predictive Check (Visual Diagnostics)

The revised model fits several aspects of the data well. The observed **overall mean** and **overall standard deviation** lie near the center of the replicated distributions, indicating good agreement in location and overall variability. The **within-game range** is also reasonably captured, with the observed value falling within the bulk of the simulated values.

However, the model still struggles with **temporal dependence**. The observed lag-1 autocorrelation lies far to the right of the replicated distribution, showing that real pitch velocities are more persistent than the model predicts. We plan to explore this idea in future iterations of this project.

Posterior Predictive Checks

Histogram = replicated seasons from posterior draws; red line = observed statistic



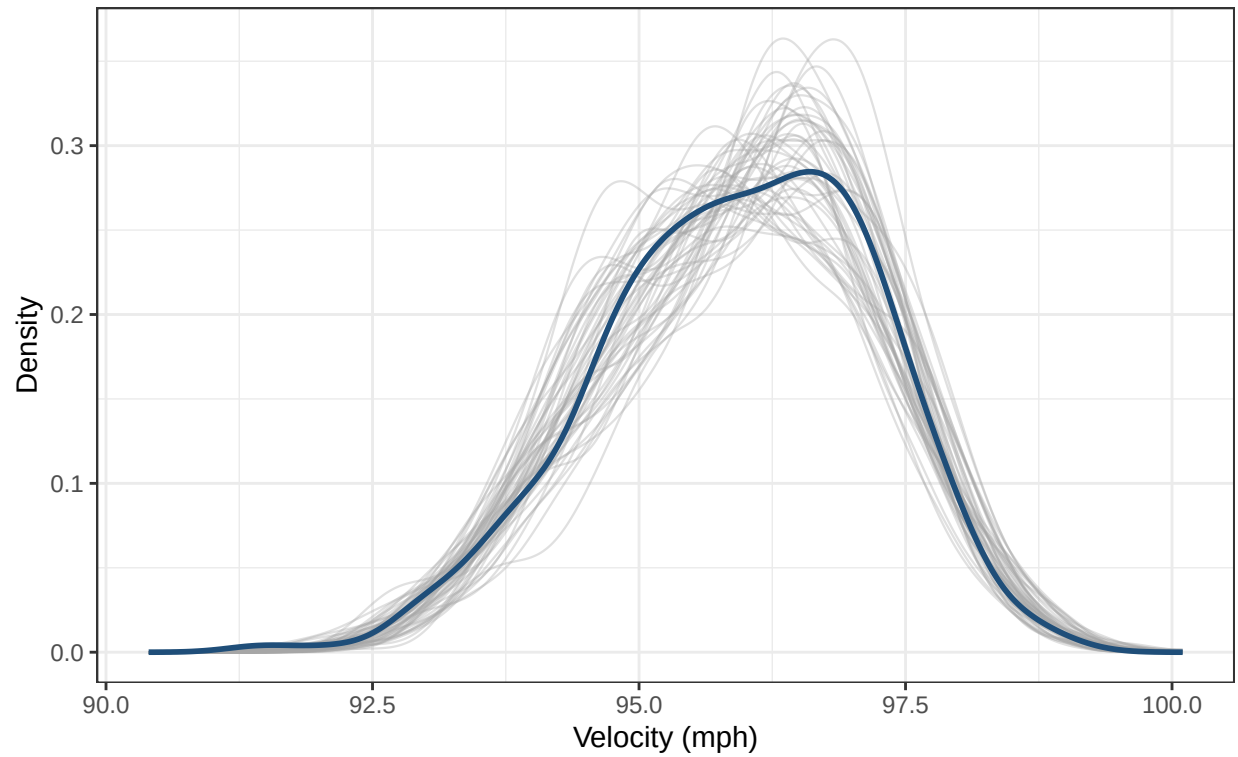
Posterior Predictive Check (Density Overlay)

Just as we did with the previous model, for visual comparison, we overlay the observed velocity density with densities from replicated seasons simulated. The observed curve lies largely within the envelope of replicated densities, indicating the model now captures the overall shape of the velocity distribution much more closely.

The fit is strongest in the center and main spread of the distribution. Some minor discrepancies remain in sharpness and tail behavior, but these are substantially smaller than before. Overall, the revised model provides a much more realistic representation of the full velocity distribution, even though some temporal dependence remains inadequately captured.

Observed vs Posterior Predictive Velocity Density

Blue = observed season; gray = replicated seasons



4. Interpretation

What Did We Learn?

The HMM identifies meaningful latent performance regimes using only velocity, with clear separation between Baseline and Hot states and strong persistence across pitches. This supports the idea that pitcher condition evolves gradually rather than randomly.

The original model, however, attributes too much structure to state switching, producing unrealistic dynamics. The revised model improves this by allowing state-specific variances and a within-game trend, separating three sources of variation: differences in mean velocity, variability within states, and systematic changes over a game. This results in a substantially better fit and more realistic pitch-to-pitch behavior.

Limitations

Despite improvements, the model remains a rough oversimplification. It relies on a single observable (velocity), ignoring other relevant signals such as spin rate, position, or pitch movement. Our model is also only used to examine the performance of one MLB pitcher, Gerritt Cole. A more complex hierarchical structure that uses partial pooling must be incorporated to apply this to multiple players. The model is also computationally intensive, as Gibbs sampling over latent states can be slow and may mix poorly.

In addition, the model ignores external context such as inning, opponent, or game conditions, which may significantly influence performance. Finally, even the revised model underestimates temporal dependence, indicating that some dynamic structure remains unexplained.

When Should We Use This Model?

This model is appropriate when data is sequential and driven by an unobserved evolving state, and when the goal is to identify latent regime. It is particularly useful when observations exhibit temporal dependence and can be grouped into independent sequences, such as games.

The original model is suitable when a simple, interpretable framework is sufficient and the goal is to detect coarse regime changes. The revised model is preferred when within-sequence trends or differences in variability are present, and when improved fit and realism are more important than simplicity.

Sensitivity to Modeling Assumptions

Inference depends on key modeling choices including the number of latent states, prior specification, and the form of the emission distribution. The assumption of two states may compress more complex behavior, while the Normal emission assumption may miss skewness or heavy tails.

The revised model introduces additional flexibility, but also potential competition between components (trend, variance, and state switching), which can complicate interpretation. These sensitivities suggest that results should be interpreted cautiously and validated through posterior predictive checks and alternative model specifications.